# Desenvolvimento de um Sistema de Gerenciamento de Banco de Dados para uma Fábrica de Veículos, Utilizando *Python* Integrado com *Sqlite*

Thiago Sales Lima (UFCG-CDSA) *thiaguinhisales@gmail.com*
Naelly Aparecida Silva Santos (UFCG-CDSA) *naelly5151@gmail.com*
Sara Jatobá Gonçalves Bezerra (UFCG-CDSA) *sarajatoba17@gmail.com*
Italo Mayacone Martins Barbosa (UFCG-CDSA) *italomayacone00@gmail.com*
Cecir Barbosa de Almeida Farias (UFCG-CDSA) *cecir.almeida@gmail.com*

**Resumo**

O crescente volume de dados na era digital demanda sistemas eficientes para armazenamento e análise de informações. Este artigo apresenta o desenvolvimento de um sistema de banco de dados centralizado para otimizar o fluxo de componentes em ambientes industriais complexos, como fábricas de veículos. O sistema proposto visa melhorar a integração entre departamentos, controlar o estoque e gerenciar fornecedores de forma eficiente, reduzindo o tempo de espera e garantindo a entrega correta dos materiais. A metodologia adotada inclui uma análise do problema, revisão bibliográfica sobre conceitos de modelagem de dados e a construção de diagramas para representar os objetivos do sistema. O desenvolvimento envolveu a implementação do banco de dados com linguagem SQL – *Structured Query Language* e a linguagem *Python* para criação das consultas e armazenamento de dados. Os resultados demonstram que o sistema desenvolvido consegue aprimorar a gestão de componentes e fornecedores, além de garantir uma operação mais ágil e eficiente na produção industrial. A pesquisa é quantitativa e exploratória, focada em analisar a eficácia do sistema na melhoria dos processos e na tomada de decisões baseadas em dados precisos.

**Palavras-Chaves:** *Sistema de Banco de Dados, MySQL, Python .*

## 1. Introdução

Atualmente, os sistemas de bancos de dados desempenham um papel fundamental na era da informação, onde a quantidade de dados gerados e armazenados cresce exponencialmente. Com o advento da tecnologia digital, empresas e instituições precisam não apenas de maneiras

eficazes de armazenar dados, mas também de ferramentas que permitam a análise rápida e a tomada de decisões informadas. Segundo Elmasri e Navathe (2016), os sistemas de bancos de dados são essenciais para gerenciar de forma eficiente grandes volumes de dados, fornecendo suporte adequado ao armazenamento, recuperação e manipulação das informações.

Em ambientes industriais complexos, como fábricas de veículos, a integração eficiente entre diferentes departamentos é essencial para o bom funcionamento da produção. Cada departamento especializado, seja na construção de *limousines*, caminhões, *vans* ou *trailers*, depende de um fluxo constante e organizado de componentes para garantir que o processo de montagem ocorra sem interrupções. A gestão eficaz desses componentes, desde o pedido até a entrega, desempenha um papel fundamental na produtividade e no cumprimento de prazos.

Normalmente, o processo de fabricação começa com o envio de um pedido de componentes pelo departamento responsável pela construção de determinado veículo. Este pedido é enviado ao departamento de compras, que verifica a disponibilidade dos itens solicitados no estoque. Para agilizar esse processo, mantém-se um estoque estratégico de componentes mais solicitados, permitindo uma entrega quase imediata dos itens. Entretanto, quando um componente não está disponível, ele deve ser encomendado a fornecedores externos, o que pode introduzir atrasos. Além disso, um mesmo item pode ter vários fornecedores, o que adiciona complexidade ao processo de aquisição e rastreamento.

Diante desses desafios, surge a necessidade de um sistema de Banco de Dados centralizado que permita o rastreamento preciso dos pedidos, o controle do estoque e a gestão eficiente dos fornecedores. Este artigo propõe o desenvolvimento de um sistema de Banco de Dados para a empresa fictícia Automata Inc. que visa otimizar o fluxo de informações e melhorar a eficiência no fornecimento de componentes, reduzindo o tempo de espera e garantindo que os materiais sejam entregues corretamente.

## 2. Referencial Teórico

### 2.1 Diagrama de Use Case

Conforme Schneider (1998), "o diagrama de Caso de Uso, formalizado na UML (*Unified Modelling Language*), concentra-se na descrição do uso de um sistema por meio de atores, sendo que um ator representa os elementos externos que interagem com o sistema". Assim, o diagrama ilustra a sequência de ações que um usuário deve seguir ao interagir com o sistema para realizar uma tarefa.

Desta forma, o diagrama permite a criação de múltiplos cenários, funcionando como uma instância de um caso de uso. Cada diagrama representa um cenário específico de interação de um ator com o sistema, e diferentes sequências podem ser seguidas durante essa interação, conforme o contexto do sistema.

## 2.2 Diagrama de Entidade e Relacionamento

O Diagrama de Entidade-Relacionamento (DER) é uma ferramenta essencial para a modelagem de dados, pois possibilita uma representação visual das entidades, atributos e relacionamentos que existem em um sistema, conforme destacado por Connolly e Begg (2014). Amplamente utilizado na criação de bases de dados, este diagrama oferece uma representação simples e organizada da estrutura dos dados. Através do DER, é possível conceber as informações essenciais de um sistema, destacando as entidades principais e as suas conexões.

No DER, as entidades correspondem a objetos do mundo real, enquanto os atributos especificam as suas propriedades. De acordo com Connolly e Begg (2014), "entidades podem ser pessoas, lugares, objetos do mundo real ou conceitos", e atributos são "características que descrevem essas entidades".

Nos diagramas E-R, os relacionamentos representam como as entidades se ligam e interagem entre si. Elmasri e Navathe (2016) afirmam que "um relacionamento é uma associação entre entidades" e que "os relacionamentos podem ser classificados quanto ao grau, cardinalidade e opção". A cardinalidade determina o número de objetos de uma entidade que podem estar associados a objetos de outra entidade, enquanto a opcionalidade indica se a participação no relacionamento é obrigatória ou facultativa.

Na prática, os Diagramas E-R são amplamente utilizados na concepção de bancos de dados relacionais, auxiliando na criação de um modelo conceitual dos dados que serve como base para os esquemas de banco de dados.

## 2.3 Sistema de Banco de Dados

Um Banco de Dados é uma tecnologia poderosa para gerenciamento de dados, que possibilita o armazenamento, manipulação e recuperação de informações em formato tabular. De acordo com Date (2014), "um banco de dados é uma coleção persistente de dados utilizada pelos

sistemas de aplicação de uma empresa, sendo o local onde são armazenadas as informações essenciais para as operações de uma organização". Ele é amplamente empregado em sistemas de gerenciamento de bancos de dados relacionais, como *Microsoft SQL Server*, *Oracle Database*, *MySQL* e *PostgreSQL*.

O *SQLite*, o SGBD selecionado para este trabalho, "é um sistema de gerenciamento de banco de dados relacional de código aberto amplamente utilizado em aplicações gratuitas para administrar suas bases de dados. Esse serviço utiliza a linguagem SQL (*Structured Query Language*), que é a linguagem mais popular para inserir, acessar e gerenciar o conteúdo armazenado em um banco de dados", conforme Pisa (2012).

Na SQL, os dados são estruturados em tabelas formadas por linhas e colunas, e as consultas são executadas utilizando a linguagem SQL para recuperar, atualizar ou inserir informações. Além disso, a SQL oferece funcionalidades avançadas para assegurar a integridade dos dados, realizar transações de forma segura e otimizar consultas para um melhor desempenho. A linguagem permite a criação e consulta das tabelas, facilitando o acesso eficiente aos dados em diversas aplicações, como sistemas de comércio eletrônico e gerenciamento de recursos humanos.

A opção pela linguagem *Python* segundo Jesus *et al* (2019) se deve a suas diversas características vantajosas. *Python* oferece uma variedade de estruturas de alto nível, como listas e dicionários, além de uma ampla gama de módulos prontos que podem ser utilizados. Adicionalmente, existem *frameworks* de terceiros que podem ser incorporados ao projeto. A linguagem é interpretada por meio de *bytecode* na máquina virtual *Python*, o que garante a portabilidade do código. Dessa forma, é possível compilar aplicações em uma plataforma e executá-las em diferentes sistemas, ou ainda rodar diretamente o código fonte. (Borges, 2010).

## 3. Metodologia

Com o objetivo de explicar de forma clara a estruturação deste trabalho, nesta seção é apresentada a metodologia utilizada para o planejamento e execução do estudo. Inicialmente, é realizada uma análise do problema para identificar os objetivos e funcionalidades da empresa. Após isso, é feito um estudo de revisão bibliográfica para se obter uma maior compreensão acerca dos conceitos de Banco de Dados, Diagrama de *Use Case*, Diagrama de Entidades e Relacionamentos (DER), Modelo Relacional (MR), posteriormente, é feita a representação

desses objetivos e funcionalidades através desses diagramas. A Figura 1 abaixo apresenta o fluxo metodológico de etapas adotadas para realização deste estudo:

**Figura 1** – Fluxo metodológico

**Fonte:** Autores (2024)

Na última parte, foram armazenados dados no banco de dados SQL utilizando linguagem de programação *Python*, e então realiza-se consultas e testes dentro do sistema através de comandos do banco de dados utilizando a linguagem de consulta estruturada SQL, este estudo é de natureza quantitativa e de caráter exploratório, e seu principal objetivo é desenvolver e analisar os dados armazenados e realizar seus variados tipos de consultas no sistema de banco de dados.

## 4. Resultados e Discussão

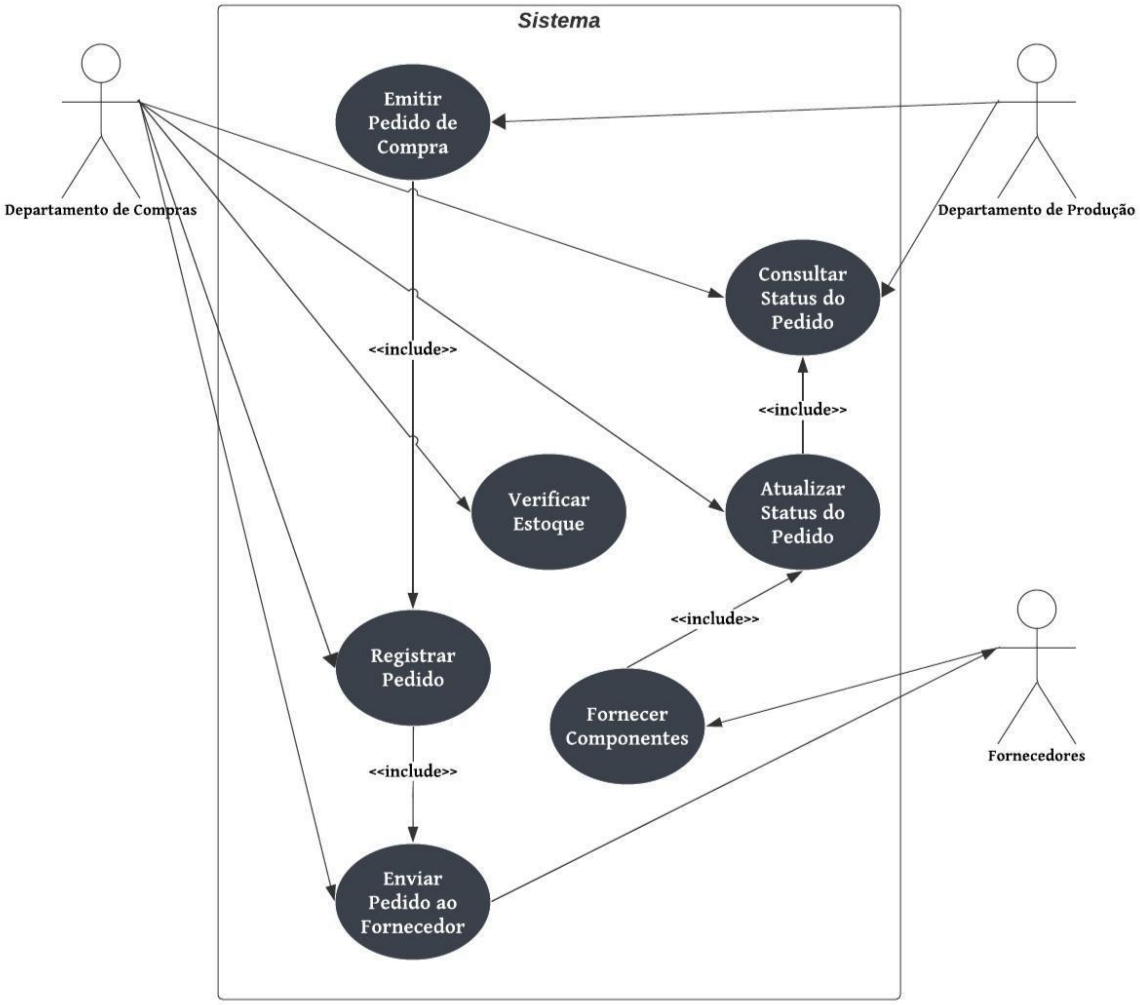
### 4.1 Objetivos e Funcionalidade do Sistema

A Automata Inc. é especializada na produção de veículos especiais sob contrato e opera diversos departamentos, cada um dedicado à construção de um tipo específico de veículo, como limousine, caminhão, van ou *trailer*. Antes de iniciar a construção de um veículo, o departamento responsável emite um pedido ao departamento de compras, especificando os componentes necessários. O departamento de compras da Automata busca criar um banco de dados eficiente para rastrear os pedidos e agilizar a entrega dos materiais.

O sistema de banco de dados foi projetado aqui para gerenciar e rastrear os pedidos recebidos, que podem incluir diversos itens diferentes. Para otimizar o processo, o sistema mantém um estoque para garantir que os itens mais frequentemente solicitados estejam disponíveis para entrega imediata. Quando um pedido chega, o sistema verifica o estoque para determinar se os itens estão disponíveis. Se algum item não estiver em estoque, o sistema será capaz de encaminhá-lo a partir de um fornecedor. Além disso, cada item pode ter múltiplos fornecedores, permitindo flexibilidade e eficiência na reposição dos itens necessários.

## 4.2 Diagrama de Use Case

**Figura 2** – Diagrama de *Use Case*




**Fonte:** Autores (2024)

**Atores:** Departamento de Compras; Departamento de Produção; Fornecedores.

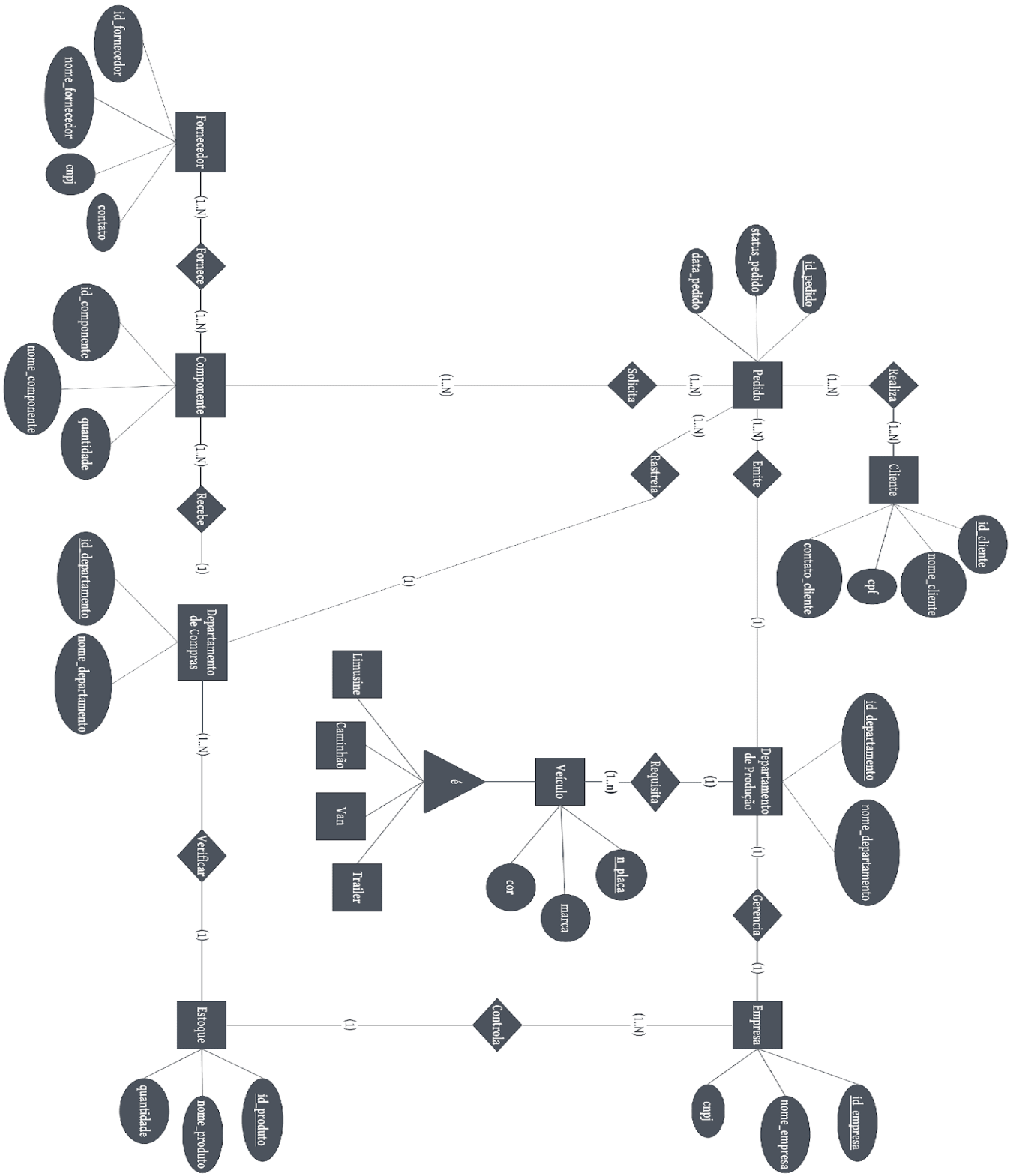
**Ações:**

a) **Departamento de Compras:** Verificar estoque, registrar pedido, enviar pedido ao fornecedor, consultar status do pedido, atualizar status do pedido.
b) **Departamento de Produção:** Emitir pedido de compra, consultar status do pedido.
c) **Fornecedores:** Fornecer componentes.

Conforme ilustrado na Figura 02, é possível identificar o Diagrama de Casos de Uso do sistema de informações da organização, o qual mostra o funcionamento das ações realizadas. O modelo delineia a sequência de atividades que ocorrem na empresa, envolvendo os diferentes atores participantes. A estrutura apresenta três atores e uma série de ações interconectadas.

### 4.3 Diagrama de Entidade e Relacionamento

**Figura 3 -** Diagrama de Entidade e Relacionamento




**Fonte:** Autores (2024)

De acordo com a Figura 03, é possível observar como ocorre o Diagrama de Entidade e Relacionamento da empresa, a forma como os mesmos estão distribuídos. O protótipo da esquematização é composto por 7 (entidades), 9 (nove) relacionamentos e os atributos

interligados em cada item necessário. Contudo, pode-se observar que na entidade cliente o Id do Cliente atua como chave primária, na entidade Pedido é o Id do Pedido, no item Departamento de Produção a chave primária é Id do Departamento, na entidade Departamento de Compras também é Id do Departamento, já no Fornecedor é Id do Fornecedor, na entidade Componente é Id do Componente, no ítem Empresa é Id da Empresa e na entidade Estoque a chave é o Id do Produto.

### 4.4 Modelo Relacional (MR)

De acordo com o modelo de entidades e relacionamentos (MER) apresentado na Figura 3, foi elaborado o Modelo Relacional, o qual possui todas as relações do sistema (descritas abaixo), que compõem as tabelas do banco de dados.

Cliente (id_cliente, nome_cliente, cpf, contato_cliente)

Pedido (id_pedido, status_pedido, data_pedido, id_departamento_compra, id_departamento_produçao)

Departamento_Producao (id_departamento, nome_departamento)

Empresa (id_empresa, nome_empresa, cnpj, id_produto)

Estoque (id_produto, nome_produto, quantidade)

Departamento_Compras (id_departamento, nome_departamento, id_produto)

Componente (id_componente, nome_componente, quantidade, id_departamento)

Fornecedor (id_fornecedor, nome_fornecedor, cnpj, contato)

Veiculo (n_placa, marca, cor, id_departamento-producao)

Cliente_Pedido (id_cliente, id_pedido)

Pedido_Componente (id_pedido, id_componente)

Componente_Fornecedor (id_componente, id_fornecedor)

Limusine (n_placa)

Caminhao (n_placa)

Van (n_placa)

Trailer (n_placa)

## 4.5 Desenvolvimento do Sistema de Banco de Dados

**Figura 4 -** Código SQL para criação do Cliente

```
1  CREATE TABLE "Cliente" (
2      "id_cliente"     INTEGER,
3      "nome_cliente"   TEXT NOT NULL,
4      "cpf"    TEXT NOT NULL,
5      "contato_cliente"    TEXT NOT NULL,
6      PRIMARY KEY("id_cliente")
7  );
```

**Fonte:** Autores (2024)

Na Figura 4 acima, encontra-se a codificação da tabela Cliente, a qual armazena informações como nome, CPF e contato. O campo id_cliente é a chave primária (*PRIMARY KEY*), identificando cada cliente de forma única. nome_cliente e cpf são obrigatórios (*NOT NULL*).

**Figura 5 -** Código SQL para criação do Pedido

```
1  CREATE TABLE "Pedido" (
2      "id_pedido" INTEGER NOT NULL,
3      "status_pedido" TEXT NOT NULL,
4      "data_pedido"   TEXT NOT NULL,
5      "id_departamento_compra"    INTEGER NOT NULL,
6      "id_departamento_producao"  INTEGER NOT NULL,
7      PRIMARY KEY("id_pedido"),
8      FOREIGN KEY("id_departamento_producao") REFERENCES "Departamento_Producao"("id_departamento"),
9      FOREIGN KEY("id_departamento_compra") REFERENCES "Departamento_Compras"("id_departamento")
10 );
```

**Fonte:** Autores (2024)

A tabela Pedido (Figura 5) registra pedidos, incluindo status e data. Ela referencia os departamentos de compras e produção via chaves estrangeiras. id_pedido é a chave primária.

**Figura 6 -** Código SQL para criação do Departamento_Producao

```
1  CREATE TABLE "Departamento_Producao" (
2      "id_departamento"    INTEGER NOT NULL,
3      "nome_departamento" TEXT NOT NULL,
4      PRIMARY KEY("id_departamento")
5  );
```

**Fonte:** Autores (2024)

A tabela Departamento_Producao (Figura6) armazena departamentos de produção. id_departamento é a chave primária, e nome_departamento é obrigatório.

**Figura 7 -** Código SQL para criação da Empresa

```
1  CREATE TABLE "Empresa" (
2      "id_empresa"    INTEGER NOT NULL,
3      "nome_empresa"  TEXT NOT NULL,
4      "cnpj"  TEXT NOT NULL,
5      "id_produto"    INTEGER NOT NULL,
6      PRIMARY KEY("id_empresa"),
7      FOREIGN KEY("id_produto") REFERENCES "Estoque"("id_produto")
8  );
```

**Fonte:** Autores (2024)

A tabela Empresa (Figura 7) contém nome, CNPJ e produtos em estoque (id_produto). id_empresa é a chave primária, e nome_empresa e CNPJ são obrigatórios.

**Figura 8 -** Código SQL para criação do Estoque

```
1  CREATE TABLE "Estoque" (
2      "id_produto"    INTEGER NOT NULL,
3      "nome_produto"  TEXT NOT NULL,
4      "quantidade"    INTEGER NOT NULL,
5      PRIMARY KEY("id_produto")
6  );
```

**Fonte:** Autores (2024)

A tabela Estoque (Figura 8) armazena produtos e suas quantidades. id_produto é a chave primária, e nome_produto e quantidade são obrigatórios.

**Figura 9 -** Código SQL para criação do Departamento_Compras

```
CREATE TABLE "Departamento_Compras" (
    "id_departamento"    INTEGER NOT NULL,
    "nome_departamento" TEXT NOT NULL,
    "id_produto"     INTEGER NOT NULL,
    PRIMARY KEY("id_departamento"),
    FOREIGN KEY("id_produto") REFERENCES "Estoque"("id_produto")
);
```

**Fonte:** Autores (2024)

A tabela Departamento_Compras (Figura 9) armazena informações dos departamentos de compras, referenciando produtos via id_produto. id_departamento e nome_departamento são obrigatórios.

**Figura 10 -** Código SQL para criação do Componente

```
CREATE TABLE "Componente" (
    "id_componente" INTEGER NOT NULL,
    "nome_componente"   TEXT NOT NULL,
    "quantidade"    INTEGER NOT NULL,
    "id_departamento_compra"    INTEGER NOT NULL,
    PRIMARY KEY("id_componente"),
    FOREIGN KEY("id_departamento_compra") REFERENCES "Departamento_Compras"("id_departamento")
);
```

**Fonte:** Autores (2024)

A tabela Componente (Figura 10) armazena componentes e quantidades, vinculando-os ao departamento de compras via id_departamento_compra.

**Figura 11 -** Código SQL para criação do Fornecedor

```
CREATE TABLE "Fornecedor" (
    "id_fornecedor" INTEGER NOT NULL,
    "nome_fornecedor"   TEXT NOT NULL,
    "cnpj"  INTEGER NOT NULL,
    "contato"   INTEGER NOT NULL,
    PRIMARY KEY("id_fornecedor")
);
```

**Fonte:** Autores (2024)

A tabela Fornecedor (Figura 11) armazena nome, CNPJ e contato. id_fornecedor é a chave primária.

**Figura 12 -** Código SQL para criação do Veiculo

```
1  CREATE TABLE "Veiculo" (
2      "n_placa"   INTEGER NOT NULL,
3      "marca" TEXT NOT NULL,
4      "cor"   TEXT NOT NULL,
5      "id_departamento"   INTEGER NOT NULL,
6      PRIMARY KEY("n_placa"),
7      FOREIGN KEY("id_departamento") REFERENCES "Departamento_Producao"("id_departamento")
8  );
```

**Fonte:** Autores (2024)

A tabela Veiculo (Figura 12) armazena marca, cor e departamento de produção (id_departamento). n_placa é a chave primária.

**Figura 13 -** Código SQL para criação da Limusine

```
1  CREATE TABLE "Limusine" (
2      "n_placa"   INTEGER NOT NULL,
3      PRIMARY KEY("n_placa")
4  );
```

**Fonte:** Autores (2024)

A tabela Limusine (Figura 13) registra limusines pelo número da placa (n_placa), que é a chave primária.

**Figura 14 -** Código SQL para criação da Caminhão

```
1  CREATE TABLE "Caminhao" (
2      "n_placa"   INTEGER NOT NULL,
3      PRIMARY KEY("n_placa")
4  );
```

**Fonte:** Autores (2024)

A tabela Caminhao (Figura 14) armazena caminhões, identificados pelo número da placa (n_placa).

**Figura 15 -** Código SQL para criação da Van

```
1  CREATE TABLE "Van" (
2      "n_placa"    INTEGER NOT NULL,
3      PRIMARY KEY("n_placa")
4  );
```

**Fonte:** Autores (2024)

A tabela Van (Figura 15) registra vans pelo número da placa, usado como chave primária.

**Figura 16 -** Código SQL para criação do Trailer

```
1  CREATE TABLE "Trailer" (
2      "n_placa"    INTEGER NOT NULL,
3      PRIMARY KEY("n_placa")
4  );
```

**Fonte:** Autores (2024)

A tabela Trailer (Figura 16) armazena trailers, identificados pela placa.

**Figura 17 -** Código SQL para criação da Cliente_Pedido

```
1  CREATE TABLE "Cliente_Pedido" (
2      "id_cliente"    INTEGER NOT NULL,
3      "id_pedido" INTEGER NOT NULL,
4      PRIMARY KEY("id_cliente","id_pedido"),
5      FOREIGN KEY("id_cliente") REFERENCES "Cliente"("id_cliente"),
6      FOREIGN KEY("id_pedido") REFERENCES "Pedido"("id_pedido")
7  );
```

**Fonte:** Autores (2024)

A tabela Cliente_Pedido (Figura 17) cria um relacionamento entre clientes e pedidos. id_cliente e id_pedido formam a chave primária composta.

**Figura 18 -** Código SQL para criação da Pedido_Componente

```sql
CREATE TABLE "Pedido_Componente" (
    "id_pedido" INTEGER NOT NULL,
    "id_componente" INTEGER NOT NULL,
    PRIMARY KEY("id_pedido","id_componente"),
    FOREIGN KEY("id_pedido") REFERENCES "Pedido"("id_pedido"),
    FOREIGN KEY("id_componente") REFERENCES "Componente"("id_componente")
);
```

**Fonte:** Autores (2024)

A tabela Pedido_Componente (Figura 18) relaciona pedidos e componentes via id_pedido e id_componente.

**Figura 19 -** Código SQL para criação da Componente_Fornecedor

```sql
CREATE TABLE "Componente_Fornecedor" (
    "id_componente" INTEGER NOT NULL,
    "id_fornecedor" INTEGER NOT NULL,
    PRIMARY KEY("id_componente","id_fornecedor"),
    FOREIGN KEY("id_fornecedor") REFERENCES "Fornecedor"("id_fornecedor"),
    FOREIGN KEY("id_componente") REFERENCES "Componente"("id_componente")
);
```

**Fonte:** Autores (2024)

A tabela Componente_Fornecedor (Figura 19) estabelece um relacionamento entre componentes e fornecedores. id_componente e id_fornecedor são chaves primárias.

## 4.6 Inserção de Dados no Banco de Dados

**Figura 20 -** Dados Inseridos na Tabela Cliente

```sql
INSERT INTO "Cliente" ("id_cliente", "nome_cliente", "cpf", "contato_cliente")
VALUES (1, 'João Silva', '123.456.789-00', 'joao@example.com');
INSERT INTO "Cliente" ("id_cliente", "nome_cliente", "cpf", "contato_cliente")
VALUES (2, 'Maria Oliveira', '987.654.321-00', 'maria@example.com');
```

**Fonte:** Autores (2024)

A tabela "Cliente" (Figura 20) armazena informações sobre os clientes da empresa, incluindo um identificador único ("id_cliente"), o nome ("nome_cliente"), CPF ("cpf") e o contato do cliente ("contato_cliente"). Dois clientes foram inseridos: João Silva com CPF 123.456.789-00

e Maria Oliveira com CPF 987.654.321-00. Ambos possuem também informações de contato, armazenadas como texto.

**Figura 21 -** Dados Inseridos na Tabela Pedido

```
INSERT INTO "Pedido" ("id_pedido", "status_pedido", "data_pedido", "id_departamento_compra", "id_departamento_producao")
VALUES (101, 'Em Andamento', '2024-10-12', 1, 1);
INSERT INTO "Pedido" ("id_pedido", "status_pedido", "data_pedido", "id_departamento_compra", "id_departamento_producao")
VALUES (102, 'Finalizado', '2024-10-13', 2, 2);
```

**Fonte:** Autores (2024)

A tabela "Pedido" criada e com dados inseridos de acordo com o código da Figura 21, armazena informações sobre os pedidos, incluindo o ID do pedido, status, data e os departamentos de compras e produção associados. Foram inseridos dois pedidos: o pedido 101, em andamento, com data de 12 de outubro de 2024, e o pedido 102, finalizado, com data de 13 de outubro de 2024.

**Figura 22 -** Dados Inseridos na Tabela Departamento_Producao

```
INSERT INTO "Departamento_Producao" ("id_departamento", "nome_departamento")
VALUES (1, 'Produção 1');
INSERT INTO "Departamento_Producao" ("id_departamento", "nome_departamento")
VALUES (2, 'Produção 2');
```

**Fonte:** Autores (2024)

A tabela "Departamento_Producao" criada e com dados inseridos de acordo com o código da da Figura 22, armazena informações sobre os departamentos de produção da empresa. Foram inseridos dois registros: Produção 1 e Produção 2, ambos com identificadores únicos ("id_departamento" 1 e 2, respectivamente).

**Figura 23 -** Dados Inseridos na Tabela Empresa

```
INSERT INTO "Empresa" ("id_empresa", "nome_empresa", "cnpj", "id_produto")
VALUES (1, 'Empresa A', '00.123.456/0001-00', 301);
INSERT INTO "Empresa" ("id_empresa", "nome_empresa", "cnpj", "id_produto")
VALUES (2, 'Empresa B', '00.987.654/0002-00', 302);
```

**Fonte:** Autores (2024)

A tabela "Empresa" criada e com dados inseridos de acordo com o código da Figura 23, armazena informações sobre as empresas fornecedoras, incluindo um identificador único

("id_empresa"), o nome da empresa, o CNPJ e o produto fornecido. As empresas "Empresa A" (id_empresa 1) e "Empresa B" (id_empresa 2) foram inseridas, associadas aos produtos 301 e 302, respectivamente.

**Figura 24 -** Dados Inseridos na Tabela Estoque

```
1   INSERT INTO "Estoque" ("id_produto", "nome_produto", "quantidade")
2   VALUES (301, 'Produto A', 500);
3   INSERT INTO "Estoque" ("id_produto", "nome_produto", "quantidade")
4   VALUES (302, 'Produto B', 1000);
```

**Fonte:** Autores (2024)

A tabela "Estoque" criada e com dados inseridos de acordo com o código da Figura 24, armazena o controle de produtos em estoque, incluindo o ID do produto, o nome do produto e a quantidade. Dois registros foram inseridos: Produto A (id_produto 301) com 500 unidades e Produto B (id_produto 302) com 1000 unidades.

**Figura 25 -** Dados Inseridos na Tabela Departamento_Compras

```
1   INSERT INTO "Departamento_Compras" ("id_departamento", "nome_departamento", "id_produto")
2   VALUES (1, 'Compras', 301);
3   INSERT INTO "Departamento_Compras" ("id_departamento", "nome_departamento", "id_produto")
4   VALUES (2, 'Aquisição', 302);
```

**Fonte:** Autores (2024)

A tabela "Departamento_Compras" criada e com dados inseridos de acordo com o código da Figura 25, armazena informações sobre os departamentos de compras da empresa. Dois departamentos foram inseridos: Compras (id_departamento 1), relacionado ao produto 301, e Aquisição (id_departamento 2), relacionado ao produto 302. A chave primária é "id_departamento".

**Figura 26 -** Dados Inseridos na Tabela Componente

```
1   INSERT INTO "Componente" ("id_componente", "nome_componente", "quantidade", "id_departamento_compra")
2   VALUES (1, 'Parafuso', 100, 1);
3   INSERT INTO "Componente" ("id_componente", "nome_componente", "quantidade", "id_departamento_compra")
4   VALUES (2, 'Porca', 200, 1);
```

**Fonte:** Autores (2024)

A tabela "Componente" criada e com dados inseridos de acordo com o código da Figura 26, armazena informações sobre os componentes utilizados pela empresa, incluindo o

"id_componente" (chave primária), o nome do componente ("nome_componente"), a quantidade em estoque e o ID do departamento de compras relacionado. Dois componentes foram inseridos: Parafuso (id_componente 1) com 100 unidades e Porca (id_componente 2) com 200 unidades, ambos vinculados ao departamento de compras 1.

**Figura 27 -** Dados Inseridos na Tabela Fornecedor

```
1   INSERT INTO "Fornecedor" ("id_fornecedor", "nome_fornecedor", "cnpj", "contato")
2   VALUES (201, 'Fornecedor A', 12345678901, 4001);
3   INSERT INTO "Fornecedor" ("id_fornecedor", "nome_fornecedor", "cnpj", "contato")
4   VALUES (202, 'Fornecedor B', 98765432101, 4002);
```

**Fonte:** Autores (2024)

A tabela "Fornecedor" criada e com dados inseridos de acordo com o código da Figura 27, armazena informações sobre os fornecedores da empresa. Dois fornecedores foram inseridos: Fornecedor A (id_fornecedor 201) com CNPJ 12345678901 e contato 4001, e Fornecedor B (id_fornecedor 202) com CNPJ 98765432101 e contato 4002.

**Figura 28 -** Dados Inseridos na Tabela Veiculo

```
1   INSERT INTO "Veiculo" ("n_placa", "marca", "cor", "id_departamento")
2   VALUES (1234, 'Fiat', 'Branco', 1);
3   INSERT INTO "Veiculo" ("n_placa", "marca", "cor", "id_departamento")
4   VALUES (5678, 'Ford', 'Preto', 2);
```

**Fonte:** Autores (2024)

A tabela "Veiculo" criada e com dados inseridos de acordo com o código da Figura 28 armazena informações sobre os veículos da empresa, incluindo o número da placa, a marca, a cor e o departamento de produção associado. Foram inseridos dois veículos: um Fiat Branco (placa 1234) e um Ford Preto (placa 5678), cada um associado a um departamento de produção.

**Figura 29 -** Dados Inseridos na Tabela Limusine

```
1   INSERT INTO "Limusine" ("n_placa") VALUES (1234);
2   INSERT INTO "Limusine" ("n_placa") VALUES (5678);
```

**Fonte:** Autores (2024)

A tabela "Limusine" criada e com dados inseridos de acordo com o código da Figura 29 armazena o número da placa das limusines. Os números de placa 1234 e 5678 foram inseridos, representando duas limusines diferentes.

**Figura 30 -** Dados Inseridos na Tabela Caminhão

```
1    INSERT INTO "Caminhao" ("n_placa") VALUES (1234);
2    INSERT INTO "Caminhao" ("n_placa") VALUES (5678);
```

**Fonte:** Autores (2024)

As instruções SQL inserem dois registros na tabela "Caminhao" (Figura 30) que armazena o número da placa de cada caminhão. A tabela possui apenas um campo, "n_placa", do tipo INTEGER, que funciona como chave primária, garantindo que cada placa de caminhão seja única. Os valores inseridos são 1234 e 5678, correspondendo a dois caminhões diferentes.

**Figura 31 -** Dados Inseridos na Tabela Van

```
1    INSERT INTO "Van" ("n_placa") VALUES (1234);
2    INSERT INTO "Van" ("n_placa") VALUES (5678);
```

**Fonte:** Autores (2024)

A tabela "Van" (Figura 31) armazena o número da placa das vans. Os números de placa 1234 e 5678 foram inseridos, representando duas vans diferentes.

**Figura 32 -** Dados Inseridos na Tabela Trailer

```
1    INSERT INTO "Trailer" ("n_placa") VALUES (1234);
2    INSERT INTO "Trailer" ("n_placa") VALUES (5678);
```

**Fonte:** Autores (2024)

A tabela "Trailer" (Figura 32) armazena o número da placa dos trailers. Os números de placa 1234 e 5678 foram inseridos, representando dois trailers diferentes.

**Figura 33 -** Dados Inseridos na Tabela Cliente_Pedido

```
1    INSERT INTO "Cliente_Pedido" ("id_cliente", "id_pedido")
2    VALUES (1, 101);
3    INSERT INTO "Cliente_Pedido" ("id_cliente", "id_pedido")
4    VALUES (2, 102);
```

**Fonte:** Autores (2024)

A tabela "Cliente_Pedido" (Figura 33) cria uma relação entre os clientes e seus pedidos, armazenando o "id_cliente" e o "id_pedido" como chaves primárias compostas. As instruções

inserem duas relações: o cliente João Silva (id_cliente 1) está associado ao pedido 101, e Maria Oliveira (id_cliente 2) ao pedido 102.

**Figura 34 -** Dados Inseridos na Tabela Pedido_Componente

```
1   INSERT INTO "Pedido_Componente" ("id_pedido", "id_componente")
2   VALUES (101, 1);
3   INSERT INTO "Pedido_Componente" ("id_pedido", "id_componente")
4   VALUES (102, 2);
```

**Fonte:** Autores (2024)

A tabela "Pedido_Componente" (Figura 34) armazena a relação entre pedidos e componentes. O pedido 101 está associado ao componente Parafuso (id_componente 1), e o pedido 102 está associado ao componente Porca (id_componente 2), usando chaves estrangeiras para vincular componentes e pedidos.

**Figura 35 -** Dados Inseridos na Tabela Componente_Fornecedor

```
1   INSERT INTO "Componente_Fornecedor" ("id_componente", "id_fornecedor")
2   VALUES (1, 201);
3   INSERT INTO "Componente_Fornecedor" ("id_componente", "id_fornecedor")
4   VALUES (2, 202);
```

**Fonte:** Autores (2024)

A tabela "Componente_Fornecedor" (Figura 35) registra a relação entre componentes e fornecedores. Dois registros foram inseridos: o componente Parafuso (id_componente 1) está relacionado ao fornecedor 201, e o componente Porca (id_componente 2) está relacionado ao fornecedor 202. Essa tabela utiliza chaves estrangeiras para vincular componentes aos fornecedores.

## 4.7 Alteração de Dados na Tabela Cliente (Comando UPDATE)

**Figura 36 -** Alteração de Dados na Tabela Cliente

```
1   UPDATE "Cliente"
2   SET "nome_cliente" = 'João Souza', "contato_cliente" = 'joao.souza@example.com'
3   WHERE "id_cliente" = 1;
4   UPDATE "Cliente"
5   SET "cpf" = '111.222.333-44', "contato_cliente" = 'maria.oliveira@example.com'
6   WHERE "id_cliente" = 2;
```

| | id_cliente | nome_cliente | cpf | contato_cliente |
|---|---|---|---|---|
| | Filtro | Filtro | Filtro | Filtro |
| 1 | 1 | João Souza | 123.456.789-00 | joao.souza@example.com |
| 2 | 2 | Maria Oliveira | 111.222.333-44 | maria.oliveira@example.com |
| 3 | 3 | Camila Ferreira | 345.678.901-23 | (31) 99876-5432 |
| 4 | 4 | Diego Costa | 456.789.012-34 | (41) 90987-6543 |
| 5 | 5 | Elisa Gonçalves | 567.890.123-45 | (51) 93456-7890 |
| 6 | 6 | Cliente C novo | 23456789012 | clienteC@example.com |
| 7 | 10 | João Silva | 123.456.789-00 | joao.silva@example.com |
| 8 | 11 | Maria Oliveira | 987.654.321-00 | maria.oliveira000@example.com |

**Fonte:** Autores (2024)

Na instrução do código SQl da Figura 36, criado para realizar atualizações nos clientes do banco de dados, os dados do cliente com id_cliente = 1 foram atualizados e "João Silva" passou a ser "João Souza", e o e-mail foi alterado para "joao.souza@example.com".

A segunda instrução modificou o CPF e o contato do cliente com id_cliente = 2, alterando o CPF para "111.222.333-44" e o e-mail para "maria.oliveira@example.com". Essas mudanças garantem a consistência das informações na tabela Cliente.

## 4.8 Consulta de Dados na Tabela Cliente (Comando SELECT)

**Figura 37 -** Consulta de Cliente pelo Nome

```
1   SELECT * FROM "Cliente"
2   WHERE "nome_cliente" = 'João Souza';
```

| | id_cliente | nome_cliente | cpf | contato_cliente |
|---|---|---|---|---|
| 1 | 1 | João Souza | 123.456.789-00 | joao.souza@example.com |

**Fonte:** Autores (2024)

A consulta SQL apresentada na Figura 37 filtra os dados da tabela Cliente pelo nome. No exemplo, busca-se "João Souza", retornando informações como id_cliente, nome_cliente, cpf e contato_cliente, sendo útil para localizar um cliente específico.

**Figura 38 -** Consulta de Cliente pelo CPF

```
1  SELECT * FROM "Cliente"
2  WHERE "cpf" = '111.222.333-44';
```

| | id_cliente | nome_cliente | cpf | contato_cliente |
|---|---|---|---|---|
| 1 | 2 | Maria Oliveira | 111.222.333-44 | maria.oliveira@example.com |

**Fonte:** Autores (2024)

A consulta SQL da Figura 38 filtra a tabela Cliente pelo CPF "111.222.333-44", retornando todas as informações associadas ao cliente. Esse tipo de consulta é útil quando o CPF é usado como identificador único.

## 4.9 Consulta entre Intervalos de Valores na Tabela Estoque (Comando SELECT com BETWEEN)

**Figura 39 -** Consulta de Estoque com Quantidades entre 100 e 300

```
1  SELECT * FROM "Estoque"
2  WHERE "quantidade" BETWEEN 100 AND 300;
```

| | id_produto | nome_produto | quantidade |
|---|---|---|---|
| 1 | 4 | Teclado Mecânico | 150 |
| 2 | 5 | Mouse Sem Fio | 300 |
| 3 | 8 | Fonte 500W | 200 |
| 4 | 9 | Memória RAM 16GB | 120 |

**Fonte:** Autores (2024)

A consulta SQL da Figura 39 retorna produtos na tabela "Estoque" cuja quantidade está entre 100 e 300 unidades, utilizando a cláusula *BETWEEN*. O resultado inclui colunas como id_produto, nome_produto e quantidade, facilitando a identificação de produtos com níveis intermediários de estoque.

**Figura 40 -** Consulta de Estoque com Quantidades entre 350 e 1000

```
1   SELECT * FROM "Estoque"
2   WHERE "quantidade" BETWEEN 350 AND 1000;
```

| | id_produto | nome_produto | quantidade |
|---|---|---|---|
| 1 | 301 | Produto A | 500 |
| 2 | 302 | Produto B | 1000 |

**Fonte:** Autores (2024)

A consulta SQL da Figura 40 busca produtos com quantidades entre 350 e 1000 unidades. Assim como a anterior, a cláusula *BETWEEN* define o intervalo desejado, retornando detalhes como id_produto, nome_produto e quantidade, permitindo visualizar produtos com maior volume em estoque.

## 4.10 Consulta Envolvendo Duas ou Três Tabelas (Comando JOIN)

**Figura 41 -** Relacionamento entre Clientes, Pedidos e Componentes

```
1   SELECT C."nome_cliente", P."id_pedido", P."status_pedido", Comp."nome_componente", Comp."quantidade"
2   FROM "Cliente" C
3   JOIN "Cliente_Pedido" CP ON C."id_cliente" = CP."id_cliente"
4   JOIN "Pedido" P ON CP."id_pedido" = P."id_pedido"
5   JOIN "Pedido_Componente" PC ON P."id_pedido" = PC."id_pedido"
6   JOIN "Componente" Comp ON PC."id_componente" = Comp."id_componente"
7   ORDER BY C."nome_cliente" ASC;
```

| | nome_cliente | id_pedido | status_pedido | nome_componente | quantidade |
|---|---|---|---|---|---|
| 1 | João Souza | 101 | Em Andamento | Parafuso | 100 |
| 2 | Maria Oliveira | 102 | Finalizado | Porca | 200 |

**Fonte:** Autores (2024)

A tabela Cliente armazena informações dos compradores, enquanto Cliente_Pedido relaciona clientes aos pedidos. A tabela Pedido contém detalhes dos pedidos, conectando-se aos departamentos de compra e produção. Já Pedido_Componente associa pedidos a componentes específicos, e a tabela Componente registra materiais utilizados, vinculando-os a pedidos e fornecedores. Essas tabelas possibilitam consultas abrangentes sobre o ciclo de pedidos (a

exemplo da consulta apresentada na Figura 41), desde o nome do cliente até os itens utilizados na produção.

### 4.11 Apagar um Registro de uma Tabela

**Figura 42 -** Exclusão de Cliente

```
1   DELETE FROM Cliente
2   WHERE id_cliente = 3;
```

| | id_cliente | nome_cliente | cpf | contato_cliente |
|---|---|---|---|---|
| | Filtro | Filtro | Filtro | Filtro |
| 1 | 1 | João Souza | 123.456.789-00 | joao.souza@example.com |
| 2 | 2 | Maria Oliveira | 111.222.333-44 | maria.oliveira@example.com |
| 3 | 4 | Diego Costa | 456.789.012-34 | (41) 90987-6543 |
| 4 | 5 | Elisa Gonçalves | 567.890.123-45 | (51) 93456-7890 |
| 5 | 6 | Cliente C novo | 23456789012 | clienteC@example.com |
| 6 | 10 | João Silva | 123.456.789-00 | joao.silva@example.com |
| 7 | 11 | Maria Oliveira | 987.654.321-00 | maria.oliveira000@example.com |

**Fonte:** Autores (2024)

O código apresentado na Figura 42 permite remover registros da tabela Cliente pelo id_cliente. No exemplo, a instrução SQL apaga o cliente com id_cliente = 3, garantindo a integridade dos dados. No entanto, essa operação deve ser realizada com cautela, pois os dados não podem ser recuperados sem um *backup*.

### 4.12 Sistema de Gerenciamento da Empresa em Django

Este sistema de gerenciamento empresarial, desenvolvido com *Django*, centraliza e facilita o controle de operações da empresa. Ele permite cadastrar, editar, consultar e excluir registros de fornecedores e clientes por meio de uma interface intuitiva. As abas "Gerenciar Fornecedores" e "Gerenciar Clientes" otimizam o cadastro e a gestão desses parceiros. O *Django* garante segurança, dinamismo e fácil integração com outras operações.

**Figura 43 –** Verificar Estoque

**Fonte:** Autores (2024)

A imagem da Figura 43 exibe um sistema de gerenciamento de estoques, permitindo verificar, pesquisar, ordenar, adicionar, editar e excluir produtos. A interface contém um cabeçalho de navegação, um formulário de busca e botões para ordenação da lista por nome ou quantidade. A tabela dinâmica exibe o estoque com opções de gerenciamento, e o sistema de *templates* do *Django* facilitam esta interação.

**Figura 44 –** Gestão de Estoque: Função de Verificação e Filtragem de Produtos no Django

```
def verificar_estoque(request):
    busca = request.GET.get('busca', '')
    ordem = request.GET.get('ordem', '')

    # Recupera todos os produtos do estoque
    produtos = Estoque.objects.all()

    # Filtra por busca
    if busca:
        produtos = produtos.filter(nome_produto__icontains=busca)

    # Ordena conforme a escolha do usuário
    if ordem == 'nome_asc':
        produtos = produtos.order_by('nome_produto')
    elif ordem == 'nome_desc':
        produtos = produtos.order_by('-nome_produto')
    elif ordem == 'quantidade_asc':
        produtos = produtos.order_by('quantidade')
    elif ordem == 'quantidade_desc':
        produtos = produtos.order_by('-quantidade')

    # Renderiza o template e passa a lista de produtos
    return render(request, 'usuarios/verificar_estoque.html', {'produtos': produtos})
```

**Fonte:** Autores (2024)

A função verificar_estoque (Figura 44) permite consultar produtos no estoque. Ela filtra e ordena os itens por nome ou quantidade e renderiza o *template* 'usuarios/verificar_estoque.html', proporcionando uma experiência interativa.

**Figura 45 –** Adição de Produtos ao Estoque: Função para Inserção no Django

```
def adicionar_estoque(request):
    if request.method == 'POST':
        nome_produto = request.POST.get('nome_produto')
        quantidade = request.POST.get('quantidade')

        if not nome_produto or not quantidade:
            return HttpResponse("Por favor, preencha todos os campos.", status=400)

        Estoque.objects.create(
            nome_produto=nome_produto,
            quantidade=quantidade
        )

        return redirect('verificar_estoque')

    return render(request, 'usuarios/adicionar_estoque.html')

```

**Fonte:** Autores (2024)

A função adicionar_estoque (Figura 46) permite incluir produtos no sistema. Quando uma requisição POST é recebida, ela extrai o nome e a quantidade do produto. Caso algum campo esteja vazio, retorna um erro. Se for válido, o produto é salvo e o usuário é redirecionado para a página de verificação.

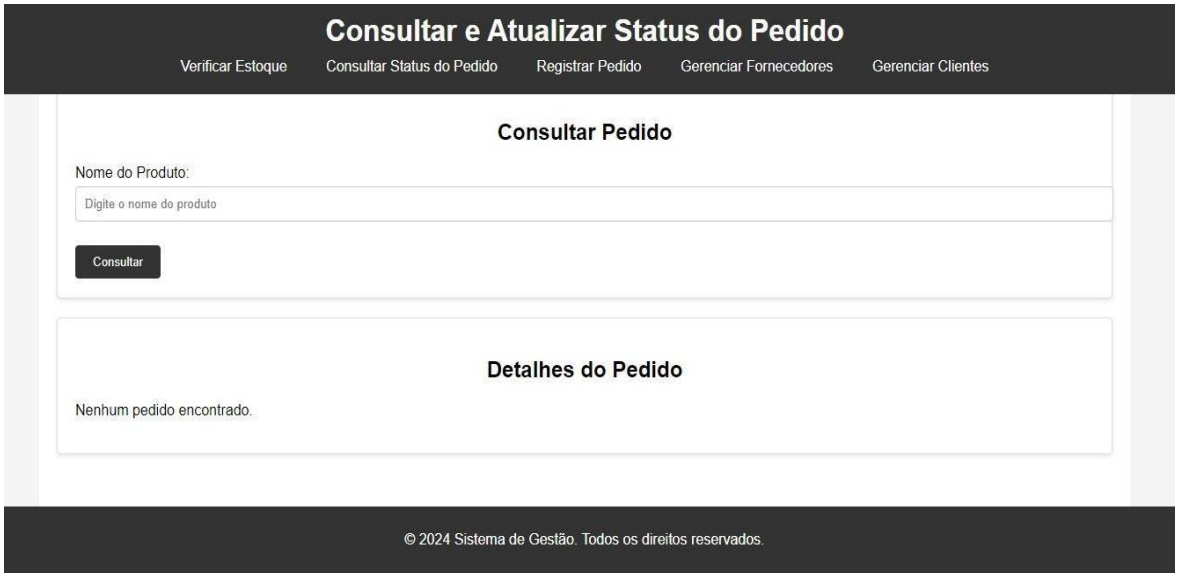
**Figura 46 –** Gerenciamento de Estoque: Funções para Edição e Exclusão de Produtos no Django

```
def editar_estoque(request, id_produto):
    estoque = get_object_or_404(Estoque, id_produto=id_produto)

    if request.method == 'POST':
        estoque.nome_produto = request.POST.get('nome', estoque.nome_produto)
        estoque.quantidade = request.POST.get('quantidade', estoque.quantidade)
        estoque.save()
        return redirect('verificar_estoque')

    return render(request, 'usuarios/editar_estoque.html', {'estoque': estoque})

def excluir_estoque(request, id_produto):
    estoque = get_object_or_404(Estoque, id_produto=id_produto)
    estoque.delete()
    return redirect('verificar_estoque')
```

**Fonte:** Autores (2024)

As funções editar_estoquee e excluir_estoquegerenciamento (Figura 46) dos produtos não estão em estoque. editar_estoquerecupera um item pelo ID e permite atualizar seus dados antes de salvar. Excluir_estoqueremova um produto do banco de dados. Ambas redirecionam para a página de verificação.

**Figura 47 –** Consultar e Atualizar Status do Pedido




**Fonte:** Autores (2024)

A funcionalidade Consultar e Atualizar *Status* do Pedido da Figura 47 permite consultar e atualizar pedidos. A interface inclui um formulário de busca e exibe detalhes como número do pedido, status, previsão de entrega, quantidade e fornecedor.

**Figura 48 –** Registrar Pedido A

**Fonte:** Autores (2024)

**Figura 49 –** Registrar Pedido B

**Fonte:** Autores (2024)

As imagens das Figuras 48 e 49 retratam as páginas onde permitem ao usuário registrar um novo pedido no sistema de gerenciamento, com campos para selecionar o cliente, fornecedor, departamento, tipo de veículo, e componente, além de informar a quantidade e os dados de entrega. Utilizando *templates Django*, a página apresenta uma interface clara e funcional que facilita a inserção dos dados necessários para o processamento de um pedido, garantindo que todas as informações relevantes sejam preenchidas corretamente antes do envio.

**Figura 50 –** Gerenciar Fornecedores

**Fonte:** Autores (2024)

A aba "Gerenciar Fornecedores" (Figura 50) permite cadastrar, editar e excluir fornecedores. A interface exibe uma tabela com os fornecedores registrados.

**Figura 51 –** Gerenciar Clientes

**Fonte:** Autores (2024)

Semelhante à aba de fornecedores, a aba "Gerenciar Clientes" (Figura 51) permite cadastrar, editar e excluir clientes. A listagem dinâmica facilita a visualização e manutenção dos registros.

**Figura 52 –** Gerenciamento de Clientes: Funções para Listagem e Cadastro no *Django*

```
31  def gerenciar_clientes(request):
32      clientes = Cliente.objects.all()
33      return render(request, 'usuarios/gerenciar_clientes.html', {'clientes': clientes})
34
35  def cadastrar_cliente(request):
36      if request.method == 'POST':
37          nome = request.POST.get('nome')
38          contato = request.POST.get('contato')
39          cpf = request.POST.get('cpf')
40
41          # Criar um novo objeto Cliente e salvar no banco de dados
42          cliente = Cliente(nome_cliente=nome, contato_cliente=contato, cpf=cpf)
43          cliente.save()
44
45          # Redireciona para a página onde a tabela de clientes é exibida
46          return redirect('gerenciar_clientes')
47
48      # Se o método não for POST, renderiza o formulário de cadastro
49      return render(request, 'usuarios/cadastrar_cliente.html')
```

**Fonte:** Autores (2024)

A função gerenciar_clientes (Figura 52) exibe todos os clientes registrados. Já cadastrar_cliente trata do cadastro de novos clientes, criando um objeto e salvando no banco de dados.

**Figura 53 –** Gerenciamento de Clientes: Funções para Edição e Exclusão no *Django*

```
# clientes/views.py
def excluir_cliente(request, id_cliente):
    cliente = get_object_or_404(Cliente, id_cliente=id_cliente)
    cliente.delete()
    return redirect('gerenciar_clientes')

def editar_cliente(request, id_cliente):
    cliente = get_object_or_404(Cliente, id_cliente=id_cliente)

    if request.method == 'POST':
        cliente.nome_cliente = request.POST.get('nome')
        cliente.contato_cliente = request.POST.get('contato')
        cliente.cpf = request.POST.get('cpf')
        cliente.save()
        return redirect('gerenciar_clientes')  # Substitua pelo nome da URL para listar clientes

    return render(request, 'usuarios/editar_cliente.html', {'cliente': cliente})
```

**Fonte:** Autores (2024)

As funções excluir_cliente e editar_cliente (Figura 53) permitem a gestão de clientes. excluir_clienter remove um registro do banco de dados. Editar_cliente permite modificar os dados e salvá-los. Ambas garantem eficiência na administração dos clientes.

## 5. Considerações Finais

O desenvolvimento de um sistema de informação e banco de dados, conforme abordado neste artigo, toma como base sistemas de gestão de banco de dados (SGBD) os quais facilitam a geração de consultas e relatórios, obtendo as informações de forma clara. O sistema atendeu aos principais objetivos estabelecidos, demonstrando ser eficiente para automatizar os processos de controle e gerenciamento de estoques. O sistema de informação e banco de dados criado oferece uma interface intuitiva para verificar estoques, consultar e analisar status do pedido, registrar pedido, gerenciar fornecedores, clientes e produtos. Através da criação dos diagramas de *Use Case* foi possível modelar adequadamente as funcionalidades do sistema, enquanto a implementação com linguagem SQL e *Python* garantiu rapidez e eficácia no gerenciamento de dados. Contudo, durante o desenvolvimento do banco de dados com SQL e *Python*, algumas dificuldades foram enfrentadas, como a integração entre o banco de dados e a aplicação *Python* exigiu ajustes para garantir a correta comunicação entre ambos, especialmente na validação de consultas mais complexas e na otimização do tempo de resposta.

Portanto, este estudo contribui para as empresas que precisam utilizar SGBDs, oferecendo um framework eficaz que pode ser aplicado a diversas indústrias com necessidades semelhantes. Pesquisas futuras podem focar na adaptação do sistema para diferentes contextos como o setor de logística e transporte, para o rastreamento de produtos e a gestão de fornecedores garantindo entregas pontuais e redução de custos operacionais. Também pode-se adaptar o sistema para o setor de saúde gerenciando medicamentos, equipamentos médicos e fornecedores de insumos, garantindo controle rigoroso sobre estoques e rastreamento de suprimentos em tempo real. Além disso, a integração com Internet das Coisas (IoT) pode ser útil para monitorar em tempo real o status dos componentes e produtos em trânsito ou armazenados, fornecendo dados precisos para o sistema para aumentar ainda mais a eficácia e robustez do sistema.

## REFERÊNCIAS

BORGES, L. E. **Python para Desenvolvedores,** 2010. Rio de Janeiro. ed.2ª. Edição do autor.

CONNOLLY, Thomas; BEG, Carolyn. **Database Systems: A Practical Approach to Design, Implementation, and Management**. 2014. Editora Addison-Wesley.

DATE, Christopher J. **Introdução a Sistemas de Banco de Dados.** 2014. Editora Campus. 1a Edição.

ELMASRI, Ramez; NAVATHE, Shamkant. **Sistemas de Banco de Dados.** Editora Pearson, 6ª edição, 2016.

JESUS, Camille; SANTANA, Bianca; BITTENCOURT, Roberto. **Oficinas de Aprendizagem de Programação com Scratch e Python em um Curso de Engenharia de Computação**. 2019. In: Anais do XXVII Workshop sobre Educação em Computação. SBC.

PISA, Pedro. O que é e como usar o MySQL? **Tech Tudo.** 2012. Abril.

SCHNEIDER, G., WINTER, J. P. **Applying Use Cases: a Practical Guide, Addison Wesley.** 1998. Yu, Eric, Modelling Strategic Relationships for Process Reengineering, Phd Thesis, University of Toronto.